

Lesson:



HashSets



Pre Requisites:

- HashMaps

List of concepts involved :

- HashTable
- HashSet
- Hierarchy of HashSet
- How to use HashSets
- Problems

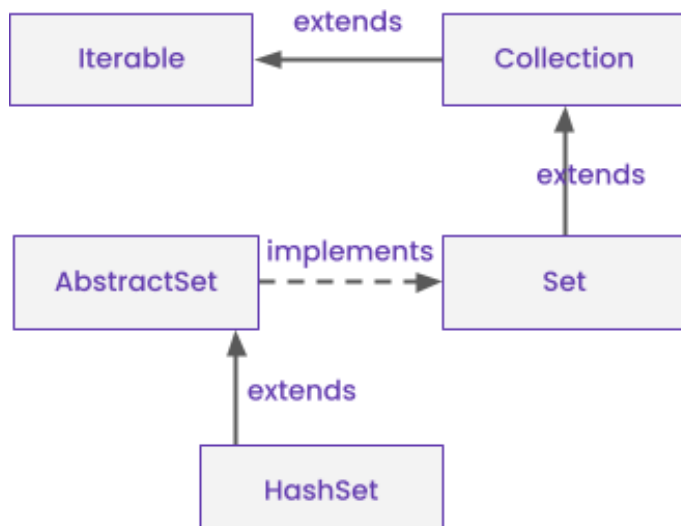
Topic: HashTable

- HashTable is like a HashMap, only synchronized and cannot hold null objects as a key or value.
- Java has provided a HashTable class inside java to implement a hash table.util package.
- Hashtable class inherits the properties of the Dictionary class and implements all the features of the Map interface. Along with this, it also implements two marker interfaces - Cloneable and Serializable.
- Whatever object is taken as the key of Hashtable is first hashed, and then the hashcode generated is used as the index at which the key value will be stored.
- To save an object and retrieve the same from the Hashtable correctly, One must override the hashcode and equals method.
- Hashtable Object creation: `Hashtable<K, V> ht = new Hashtable<K, V>();`
- The average lookup time for an element in the hash table is similar to an array, i.e. $O(1)$, irrespective of the number of items present in the data structure, while the worst-case look up time is $O(n)$.

Topic: HashSet

- Java HashSet class internally uses HashTable to implement its concept of holding unique elements.
- It internally stores elements with the help of Hashing Mechanism.
- HashSet is a non - synchronized class which can hold one null element.
- Insertion order of the elements is never maintained.
- Declaration of HashSet class-`public class HashSet<E> extends AbstractSet<E> implements Set<E>, Cloneable, Serializable {}`
- HashSet contains unique elements only.
- It stores only the unique key values and not pairs.

Topic: Hierarchy of HashSet



- HashSet class extends the features of AbstractSet class which implements Set interface.
- The Set interface inherits Collection interface in hierarchical order.

Topic: How to use HashSets

- **Syntax to declare a HashSet:**

```
HashSet<DataType> set = new HashSet<>();
```

- **Add an element to HashSet:**

Output:

<https://pastebin.com/PizH3g1T>

Time Complexity: $O(1)$

- **Check if set contains a particular key**

<https://pastebin.com/Vh2PqT8z>

Output:

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/
true

Process finished with exit code 0
```

Output: true

Time Complexity: $O(1)$

- **Size of set**

<https://pastebin.com/hvfvJteU>

Output:

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/
5

Process finished with exit code 0
```

Time Complexity: $O(1)$

- **To remove an element from the set**

<https://pastebin.com/01AnaHPM>

Output:

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/
4

Process finished with exit code 0
```

Time Complexity: $O(1)$

- Traversing through the HashSet
<https://pastebin.com/aFtnC7Bg>
 Output:

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home/bin/java
Five
One
Two
Three

Process finished with exit code 0
```

Time Complexity: $O(n)$ where size of set = n.

Problem 1:

Write a program in java to store names in a set and check whether the name entered is present or not?

CODE:
<https://pastebin.com/mVxNisVW>

Output:

```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home/bin/java
Rohit
Shivani
Ekta
Rudra
Ankit
An initial list of elements: [Rohit, Shivani, Ekta, Rudra, Ankit]
check if ekta is present in the set: true

Process finished with exit code 0
```

Problem 1:

Q2. Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

Input1:
 n = 6
 nums = [100,4,200,1,3,2]

Output1:
 4

Input2:
 n = 10
 nums = [0,3,7,2,5,8,4,6,0,1]

Output2:
 9

Explanation:

- We solve this problem using a HashSet which holds Integer values.
- First input all the elements of the array into the HashSet.
- Next, traverse over the array, and for every element, we check if the number before it is included in the HashSet or not.
- If it is not, this might be the start of a new consecutive subarray and we update the length of longest sequence as 1 and the start element of the longest sequence as the current element.
- Now, in this if condition only, we run a while loop until we can find the next consecutive element.
- For every consecutive element we find, update start of a new consecutive subarray and the length of longest sequence.
- Keep another pointer to store the longest subarray, and update it with the longest one found so far after every iteration in which we calculate length of each consecutive subarray.

Code:

<https://pastebin.com/tJiVLd4R>

```
/Library/Java/JavaVirtualMachines/jdk-1
Enter the length of the array:
6
Enter the elements of the array:
100 4 200 1 3 2
4
Process finished with exit code 0
```

Problem 3:

Q3. You are given n distinct pairs. Each pair is numbered from 1 to n. All these pairs are initially put in a bag. You need to pair up each number. You take numbers one by one from the bag and for each number you look whether the pair of this number has already been taken out of the bag, or not. If not (that means the pair of this number is still in the bag), you put the current number on the table in front of him. Otherwise, you put both numbers from the pair aside. Print the maximum number of numbers that were on the table at the same time. [Medium]

Input1:

1
11

Output1:

1

Input2:

3
2 1 3 2 3

Output2:

2

Explanation:

- The numbers on the table have to be unique, hence, we assume our table to be a HashSet.
- Now we traverse over the numbers, one by one.
- For each number, we check if it is present in the set or not. Here, we are basically checking if its pair is present in the set or not.
- If it is present in the set, i.e. taken out of the bag, we have found a pair, so remove the number from the set.
- If not, we put this number in the set to find its pair from the bag.
- Maximum number of pairs on the table represent the size of the set. Therefore, we keep checking the size of the set after every addition and keep track of the maximum size of the set.

Code:

<https://pastebin.com/NFfEAfLU>

```
/Library/Java/JavaVirtualMachines/jdk-19.j
Enter the number of pairs:
3
Enter the numbers:
2 1 1 3 2 3
2

Process finished with exit code 0
```

Upcoming Class Teasers:

- Backtracking